

Programming Assignment: Unit 6

Hset Paing Htoo

Introduction to Computer Science Program, University of the People

CS 1102-01 Programming 1 AY2026-T5

Instructor : Sirajo Musa (Instructor)

July 29, 2026

Vehicle Rental Information System

1. Introduction

This program models a car rental agency's Vehicle Information System. The agency rents three kinds of vehicles — cars, motorcycles, and trucks — and every vehicle needs to report its make, model, and year of manufacture. Each vehicle type also has its own extra details: a car has a door count and fuel type, a motorcycle has a wheel count and motorcycle type, and a truck has a cargo capacity and transmission type.

Java only allows a class to extend one superclass, but a class can implement as many interfaces as needed. That is exactly the situation here: Car, Motorcycle, and Truck all need to share the general "Vehicle" contract, but each also needs its own specialised contract. The design therefore uses one shared interface (Vehicle) plus three type-specific interfaces (CarVehicle, MotorVehicle, TruckVehicle), and each concrete class implements both the shared interface and its own specialised one.

The program consists of four interfaces, three classes that implement them, and a menu-driven main program that lets the user add vehicles and display all of them. Each section below is documented with Javadoc comments in the code and an explanation of the design decisions.

2. Interface Design

Four interfaces define the contracts used throughout the system. Vehicle is the shared contract every vehicle type must satisfy. CarVehicle, MotorVehicle, and TruckVehicle each add the methods specific to one vehicle type. Splitting the contract this way keeps each interface focused on a single responsibility, and lets a class mix in only the extra behaviour it actually needs.

2.1 Vehicle interface

Declares the three methods every vehicle type must provide: `getMake()`, `getModel()`, and `getYear()`.

Vehicle.java

```
/**
 * The Vehicle interface defines the common contract that every vehicle
 * type in the rental system must follow. Any class that implements this
 * interface guarantees it can report its make, model, and year of
 * manufacture, regardless of what specific kind of vehicle it is.
 *
 * Using an interface here (rather than a shared superclass) means Car,
 * Motorcycle, and Truck can each also implement their own specialised
 * interface (CarVehicle, MotorVehicle, TruckVehicle) at the same time,
 * since Java allows a class to implement multiple interfaces but extend
 * only one class.
 */
public interface Vehicle {

    /**
     * Returns the manufacturer of the vehicle.
     * @return the vehicle's make (e.g. "Toyota")
     */
    String getMake();

    /**
     * Returns the model name of the vehicle.
     * @return the vehicle's model (e.g. "Corolla")
     */
    String getModel();

    /**
     * Returns the year the vehicle was manufactured.
     * @return the year of manufacture
     */
    int getYear();
}
```

2.2 CarVehicle interface

Adds car-specific behaviour: setting/getting the number of doors and the fuel type.

CarVehicle.java

```
/**
 * The CarVehicle interface defines behaviour specific to cars: setting
 * and retrieving the number of doors and the fuel type. A class that
 * implements both Vehicle and CarVehicle is guaranteeing it can behave
 * like a general vehicle AND provide car-specific details.
 */
public interface CarVehicle {

    /**
     * Sets the number of doors on the car.
     * @param numDoors the number of doors (expected to be positive)
     */
    void setNumDoors(int numDoors);

    /**
```

```

    * Returns the number of doors on the car.
    * @return the number of doors
    */
    int getNumDoors();

    /**
     * Sets the car's fuel type.
     * @param fuelType one of "Petrol", "Diesel", or "Electric"
     */
    void setFuelType(String fuelType);

    /**
     * Returns the car's fuel type.
     * @return the fuel type
     */
    String getFuelType();
}

```

2.3 MotorVehicle interface

Adds motorcycle-specific behaviour: setting/getting the number of wheels and the motorcycle type.

MotorVehicle.java

```

/**
 * The MotorVehicle interface defines behaviour specific to motorcycles:
 * setting and retrieving the number of wheels and the motorcycle type.
 */
public interface MotorVehicle {

    /**
     * Sets the number of wheels on the motorcycle.
     * @param numWheels the number of wheels (usually 2 or 3)
     */
    void setNumWheels(int numWheels);

    /**
     * Returns the number of wheels on the motorcycle.
     * @return the number of wheels
     */
    int getNumWheels();

    /**
     * Sets the type of motorcycle.
     * @param motorcycleType one of "Sport", "Cruiser", or "Off-road"
     */
    void setMotorcycleType(String motorcycleType);

    /**
     * Returns the type of motorcycle.
     * @return the motorcycle type
     */
    String getMotorcycleType();
}

```

2.4 TruckVehicle interface

Adds truck-specific behaviour: setting/getting the cargo capacity and the transmission type.

TruckVehicle.java

```
/**
 * The TruckVehicle interface defines behaviour specific to trucks:
 * setting and retrieving the cargo capacity and the transmission type.
 */
public interface TruckVehicle {

    /**
     * Sets the truck's cargo capacity.
     * @param cargoCapacity the cargo capacity, in tons
     */
    void setCargoCapacity(double cargoCapacity);

    /**
     * Returns the truck's cargo capacity.
     * @return the cargo capacity, in tons
     */
    double getCargoCapacity();

    /**
     * Sets the truck's transmission type.
     * @param transmissionType one of "Manual" or "Automatic"
     */
    void setTransmissionType(String transmissionType);

    /**
     * Returns the truck's transmission type.
     * @return the transmission type
     */
    String getTransmissionType();
}
```

3. Class Implementation

Car, Motorcycle, and Truck each implement Vehicle together with their own specialised interface. Every class stores its own fields, implements every method required by both interfaces it signed up for, and overrides toString() so that displaying a vehicle automatically prints the right details for its type — this is where polymorphism shows up in the class design, not just the main program.

3.1 Car class

Implements Vehicle and CarVehicle. Stores make, model, year, number of doors, and fuel type.

Car.java

```
/**
 * Car represents a car available for rental. It implements both the
 * general Vehicle contract and the car-specific CarVehicle contract,
 * so it must provide make/model/year details as well as door count
 * and fuel type.
 */
public class Car implements Vehicle, CarVehicle {

    private String make;
    private String model;
    private int year;
    private int numDoors;
    private String fuelType;

    /**
     * Constructs a Car with its basic vehicle details. Door count and
     * fuel type are set afterwards via setNumDoors/setFuelType, since
     * those come from the CarVehicle interface.
     *
     * @param make the manufacturer, must not be null or blank
     * @param model the model name, must not be null or blank
     * @param year the year of manufacture, must be a reasonable positive year
     */
    public Car(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }

    @Override
    public String getMake() {
        return make;
    }

    @Override
    public String getModel() {
        return model;
    }

    @Override
    public int getYear() {
        return year;
    }
}
```

```

    }

    @Override
    public void setNumDoors(int numDoors) {
        this.numDoors = numDoors;
    }

    @Override
    public int getNumDoors() {
        return numDoors;
    }

    @Override
    public void setFuelType(String fuelType) {
        this.fuelType = fuelType;
    }

    @Override
    public String getFuelType() {
        return fuelType;
    }

    /**
     * Returns a human-readable summary of the car's details.
     */
    @Override
    public String toString() {
        return String.format(
            "Car -> Make: %s | Model: %s | Year: %d | Doors: %d | Fuel Type: %s",
            make, model, year, numDoors, fuelType);
    }
}

```

3.2 Motorcycle class

Implements Vehicle and MotorVehicle. Stores make, model, year, number of wheels, and motorcycle type.

Motorcycle.java

```

/**
 * Motorcycle represents a motorcycle available for rental. It implements
 * both the general Vehicle contract and the motorcycle-specific
 * MotorVehicle contract.
 */
public class Motorcycle implements Vehicle, MotorVehicle {

    private String make;
    private String model;
    private int year;
    private int numWheels;
    private String motorcycleType;

    /**
     * Constructs a Motorcycle with its basic vehicle details.
     *
     * @param make the manufacturer
     * @param model the model name
     * @param year the year of manufacture
     */
    public Motorcycle(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }
}

```

```

    }

    @Override
    public String getMake() {
        return make;
    }

    @Override
    public String getModel() {
        return model;
    }

    @Override
    public int getYear() {
        return year;
    }

    @Override
    public void setNumWheels(int numWheels) {
        this.numWheels = numWheels;
    }

    @Override
    public int getNumWheels() {
        return numWheels;
    }

    @Override
    public void setMotorcycleType(String motorcycleType) {
        this.motorcycleType = motorcycleType;
    }

    @Override
    public String getMotorcycleType() {
        return motorcycleType;
    }

    /**
     * Returns a human-readable summary of the motorcycle's details.
     */
    @Override
    public String toString() {
        return String.format(
            "Motorcycle -> Make: %s | Model: %s | Year: %d | Wheels: %d | Type: %s",
            make, model, year, numWheels, motorcycleType);
    }
}

```

3.3 Truck class

Implements Vehicle and TruckVehicle. Stores make, model, year, cargo capacity, and transmission type.

Truck.java

```

/**
 * Truck represents a truck available for rental. It implements both the
 * general Vehicle contract and the truck-specific TruckVehicle contract.
 */
public class Truck implements Vehicle, TruckVehicle {

    private String make;
    private String model;
    private int year;

```



```

private double cargoCapacity;
private String transmissionType;

/**
 * Constructs a Truck with its basic vehicle details.
 *
 * @param make the manufacturer
 * @param model the model name
 * @param year the year of manufacture
 */
public Truck(String make, String model, int year) {
    this.make = make;
    this.model = model;
    this.year = year;
}

@Override
public String getMake() {
    return make;
}

@Override
public String getModel() {
    return model;
}

@Override
public int getYear() {
    return year;
}

@Override
public void setCargoCapacity(double cargoCapacity) {
    this.cargoCapacity = cargoCapacity;
}

@Override
public double getCargoCapacity() {
    return cargoCapacity;
}

@Override
public void setTransmissionType(String transmissionType) {
    this.transmissionType = transmissionType;
}

@Override
public String getTransmissionType() {
    return transmissionType;
}

/**
 * Returns a human-readable summary of the truck's details.
 */
@Override
public String toString() {
    return String.format(
        "Truck -> Make: %s | Model: %s | Year: %d | Cargo Capacity: %.1f tons | Transmission: %s",
        make, model, year, cargoCapacity, transmissionType);
}
}

```

4. Main Program

VehicleRentalApp is a console, menu-driven program. The user repeatedly chooses to add a Car, Motorcycle, or Truck, or to display every vehicle added so far, until they choose to exit. All vehicles — regardless of their concrete type — are stored in a single `List<Vehicle>`. This is only possible because of polymorphism: since Car, Motorcycle, and Truck all implement Vehicle, a variable of type Vehicle can refer to any of them, and calling `toString()` on each element automatically runs that object's own version, printing its type-specific details without the display method needing to check what subtype it is.

VehicleRentalApp.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

/**
 * VehicleRentalApp is the main program for the car rental agency's
 * vehicle information system. It lets the user interactively create
 * Car, Motorcycle, and Truck objects, storing each one as a Vehicle,
 * and then display the details of every vehicle that has been added.
 *
 * All vehicles are stored in a single List<Vehicle> even though they
 * are three different concrete classes -- this is polymorphism in
 * action: the list only needs to know each element is "a Vehicle",
 * and each object's toString() prints its own subtype-specific details.
 */
public class VehicleRentalApp {

    private static final Scanner scanner = new Scanner(System.in);
    private static final List<Vehicle> vehicles = new ArrayList<>();

    public static void main(String[] args) {
        boolean running = true;

        System.out.println("=====");
        System.out.println(" Car Rental Agency - Vehicle Info System");
        System.out.println("=====");

        while (running) {
            printMenu();
            int choice = readInt("Enter your choice: ", 1, 5);

            switch (choice) {
                case 1:
                    addCar();
                    break;
                case 2:
                    addMotorcycle();
                    break;
                case 3:
                    addTruck();
                    break;
                case 4:
                    displayAllVehicles();
            }
        }
    }
}
```

```

        break;
    case 5:
        running = false;
        System.out.println("Goodbye!");
        break;
    default:
        // readInt already restricts the range, so this is unreachable,
        // but it is kept as a defensive fallback.
        System.out.println("Invalid choice.");
    }
}

scanner.close();
}

private static void printMenu() {
    System.out.println();
    System.out.println("1. Add a Car");
    System.out.println("2. Add a Motorcycle");
    System.out.println("3. Add a Truck");
    System.out.println("4. Display all vehicles");
    System.out.println("5. Exit");
}

/**
 * Prompts for and creates a new Car, then adds it to the vehicle list.
 */
private static void addCar() {
    System.out.println("\n-- New Car --");
    String make = readNonEmptyString("Make: ");
    String model = readNonEmptyString("Model: ");
    int year = readInt("Year of manufacture: ", 1900, 2100);

    Car car = new Car(make, model, year);
    car.setNumDoors(readInt("Number of doors: ", 1, 10));
    car.setFuelType(readChoice("Fuel type (Petrol/Diesel/Electric): ",
        new String[]{"Petrol", "Diesel", "Electric"}));

    vehicles.add(car);
    System.out.println("Car added successfully.");
}

/**
 * Prompts for and creates a new Motorcycle, then adds it to the vehicle list.
 */
private static void addMotorcycle() {
    System.out.println("\n-- New Motorcycle --");
    String make = readNonEmptyString("Make: ");
    String model = readNonEmptyString("Model: ");
    int year = readInt("Year of manufacture: ", 1900, 2100);

    Motorcycle motorcycle = new Motorcycle(make, model, year);
    motorcycle.setNumWheels(readInt("Number of wheels: ", 2, 4));
    motorcycle.setMotorcycleType(readChoice("Motorcycle type (Sport/Cruiser/Off-road): ",
        new String[]{"Sport", "Cruiser", "Off-road"}));

    vehicles.add(motorcycle);
    System.out.println("Motorcycle added successfully.");
}

/**
 * Prompts for and creates a new Truck, then adds it to the vehicle list.
 */
private static void addTruck() {
    System.out.println("\n-- New Truck --");
    String make = readNonEmptyString("Make: ");
    String model = readNonEmptyString("Model: ");
    int year = readInt("Year of manufacture: ", 1900, 2100);

```

```

        Truck truck = new Truck(make, model, year);
        truck.setCargoCapacity(readDouble("Cargo capacity (tons): ", 0, 200));
        truck.setTransmissionType(readChoice("Transmission type (Manual/Automatic): ",
            new String[]{"Manual", "Automatic"}));

        vehicles.add(truck);
        System.out.println("Truck added successfully.");
    }

    /**
     * Displays every vehicle added so far by relying on polymorphism:
     * each vehicle's own toString() is invoked automatically, so this
     * method does not need to know or check which concrete class each
     * vehicle actually is.
     */
    private static void displayAllVehicles() {
        System.out.println();
        if (vehicles.isEmpty()) {
            System.out.println("No vehicles have been added yet.");
            return;
        }

        System.out.println("---- All Vehicles (" + vehicles.size() + ") ----");
        for (int i = 0; i < vehicles.size(); i++) {
            System.out.println((i + 1) + ". " + vehicles.get(i));
        }
    }

    // -----
    // Input helpers -- all user input funnels through these so that
    // invalid input (wrong type, out-of-range value, blank text, or a
    // fuel/motorcycle/transmission type that isn't one of the allowed
    // options) is caught and re-prompted instead of crashing the program.
    // -----

    /**
     * Reads an integer within [min, max] from the user, re-prompting on
     * non-numeric input or out-of-range values.
     */
    private static int readInt(String prompt, int min, int max) {
        while (true) {
            System.out.print(prompt);
            try {
                int value = Integer.parseInt(scanner.nextLine().trim());
                if (value < min || value > max) {
                    System.out.printf("Please enter a number between %d and %d.%n", min, max);
                    continue;
                }
                return value;
            } catch (NumberFormatException e) {
                System.out.println("That's not a valid whole number. Please try again.");
            }
        }
    }

    /**
     * Reads a double within [min, max] from the user, re-prompting on
     * non-numeric input or out-of-range values.
     */
    private static double readDouble(String prompt, double min, double max) {
        while (true) {
            System.out.print(prompt);
            try {
                double value = Double.parseDouble(scanner.nextLine().trim());
                if (value < min || value > max) {
                    System.out.printf("Please enter a number between %.1f and %.1f.%n", min, max);
                    continue;
                }
                return value;
            }
        }
    }

```

```

        } catch (NumberFormatException e) {
            System.out.println("That's not a valid number. Please try again.");
        }
    }
}

/**
 * Reads a non-blank line of text from the user, re-prompting if the
 * input is empty.
 */
private static String readNonEmptyString(String prompt) {
    while (true) {
        System.out.print(prompt);
        String value = scanner.nextLine().trim();
        if (value.isEmpty()) {
            System.out.println("This field cannot be empty. Please try again.");
            continue;
        }
        return value;
    }
}

/**
 * Reads a string from the user and validates it against a fixed set
 * of allowed options (case-insensitive), re-prompting until the
 * input matches one of them.
 */
private static String readChoice(String prompt, String[] options) {
    while (true) {
        System.out.print(prompt);
        String value = scanner.nextLine().trim();
        for (String option : options) {
            if (option.equalsIgnoreCase(value)) {
                return option;
            }
        }
        System.out.println("Invalid option. Please enter one of the listed choices.");
    }
}
}

```

5. Error Handling

All user input is funneled through four helper methods so that invalid input is handled in one place rather than repeated at every prompt:

- `readInt(prompt, min, max)` — rejects non-numeric text and values outside the allowed range, re-prompting until a valid whole number is entered.
- `readDouble(prompt, min, max)` — same idea, for decimal values such as cargo capacity.
- `readNonEmptyString(prompt)` — rejects blank input for text fields like make and model.
- `readChoice(prompt, options)` — only accepts one of a fixed set of options (e.g. "Petrol", "Diesel", "Electric"), case-insensitively, re-prompting on anything else.

Because input is read as a full line and parsed manually (`Integer.parseInt` / `Double.parseDouble` inside a `try/catch`), the program catches `NumberFormatException` instead of crashing when the user types something like letters where a number is expected, and simply asks again. Every add-vehicle flow and the main menu choice itself goes through these helpers, so the program cannot be crashed by malformed input.

6. Sample Output

The transcript below is from a test run that exercises the error-handling paths on purpose (an invalid year, an invalid fuel type, and an invalid motorcycle type are entered first and rejected) before adding one Car, one Motorcycle, and one Truck, and displaying all three.

```
hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-6
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-6$ javac *.java
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-6$ java VehicleRentalApp
=====
Car Rental Agency - Vehicle Info System
=====
1. Add a Car
2. Add a Motorcycle
3. Add a Truck
4. Display all vehicles
5. Exit
Enter your choice: 1

-- New Car --
Make: Toyota
Model: Corolla
Year: abc
That's not a valid whole number. Please try again.
Year: 2022
Number of doors: 4
Fuel Type (Petrol/Diesel/Electric): Gasoline
Invalid option. Please enter one of the listed choices.
Fuel Type (Petrol/Diesel/Electric): Petrol
Car added successfully!

1. Add a Car
2. Add a Motorcycle
3. Add a Truck
4. Display all vehicles
5. Exit
Enter your choice: 2
```

```
hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-6
1. Add a Car
2. Add a Motorcycle
3. Add a Truck
4. Display all vehicles
5. Exit
Enter your choice: 2

-- New Motorcycle --
Make: Yamaha
Model: MT-07
Year: 2021
Number of wheels: 2
Motorcycle type (Sport/Cruiser/Off-road): Cruise
Invalid option. Please enter one of the listed choices.
Motorcycle type (Sport/Cruiser/Off-road): Sport
Motorcycle added successfully!

1. Add a Car
2. Add a Motorcycle
3. Add a Truck
4. Display all vehicles
5. Exit
Enter your choice: 3

-- New Truck --
Make: Volvo
Model: FH16
Year: 2023
Cargo capacity (in tons): 25
Transmission type (Manual/Automatic): Manual
Truck added successfully!
```

```
hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-6
5. Exit
Enter your choice: 3

-- New Truck --
Make: Volvo
Model: FH16
Year: 2023
Cargo capacity (in tons): 25
Transmission type (Manual/Automatic): Manual
Truck added successfully!

1. Add a Car
2. Add a Motorcycle
3. Add a Truck
4. Display all vehicles
5. Exit
Enter your choice: 4

---- All Vehicles (3) ----
1. Car -> Make: Toyota | Model: Corolla | Year: 2022 | Doors: 4 | Fuel Type: Petrol
2. Motorcycle -> Make: Yamaha | Model: MT-07 | Year: 2021 | Wheels: 2 | Type: Sport
3. Truck -> Make: Volvo | Model: FH16 | Year: 2023 | Cargo Capacity: 25.0 tons | Transmission: Manual

1. Add a Car
2. Add a Motorcycle
3. Add a Truck
4. Display all vehicles
5. Exit
Enter your choice: 5
Exiting the application. Goodbye!
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-6$ |
```

7. Conclusion

This assignment demonstrates interfaces as contracts (Vehicle, CarVehicle, MotorVehicle, TruckVehicle), multiple interface implementation as Java's substitute for multiple inheritance (each class implements two interfaces at once), and polymorphism (a single List<Vehicle> holding three different concrete types, each displaying itself correctly through toString()). Input validation keeps the program robust against invalid user input, and Javadoc comments document every interface, class, and method.